状态压缩动态规划（State Compression DP）

一、基本概念

1.1 什么是状态压缩DP

状态压缩动态规划是一种**用二进制数（或其他压缩方式）表示状态集合**的动态规划技巧，主要用于解决状态空间与**集合选取**相关的问题。

1.2 适用场景

- **状态维度多**但每维状态值少（通常是二进制选择）
- **小规模**的集合选取问题（n ≤ 20-30）
- 状态间**依赖关系**可以通过位运算高效计算
- 典型问题：旅行商问题、棋盘覆盖、任务分配等

1.3 核心思想

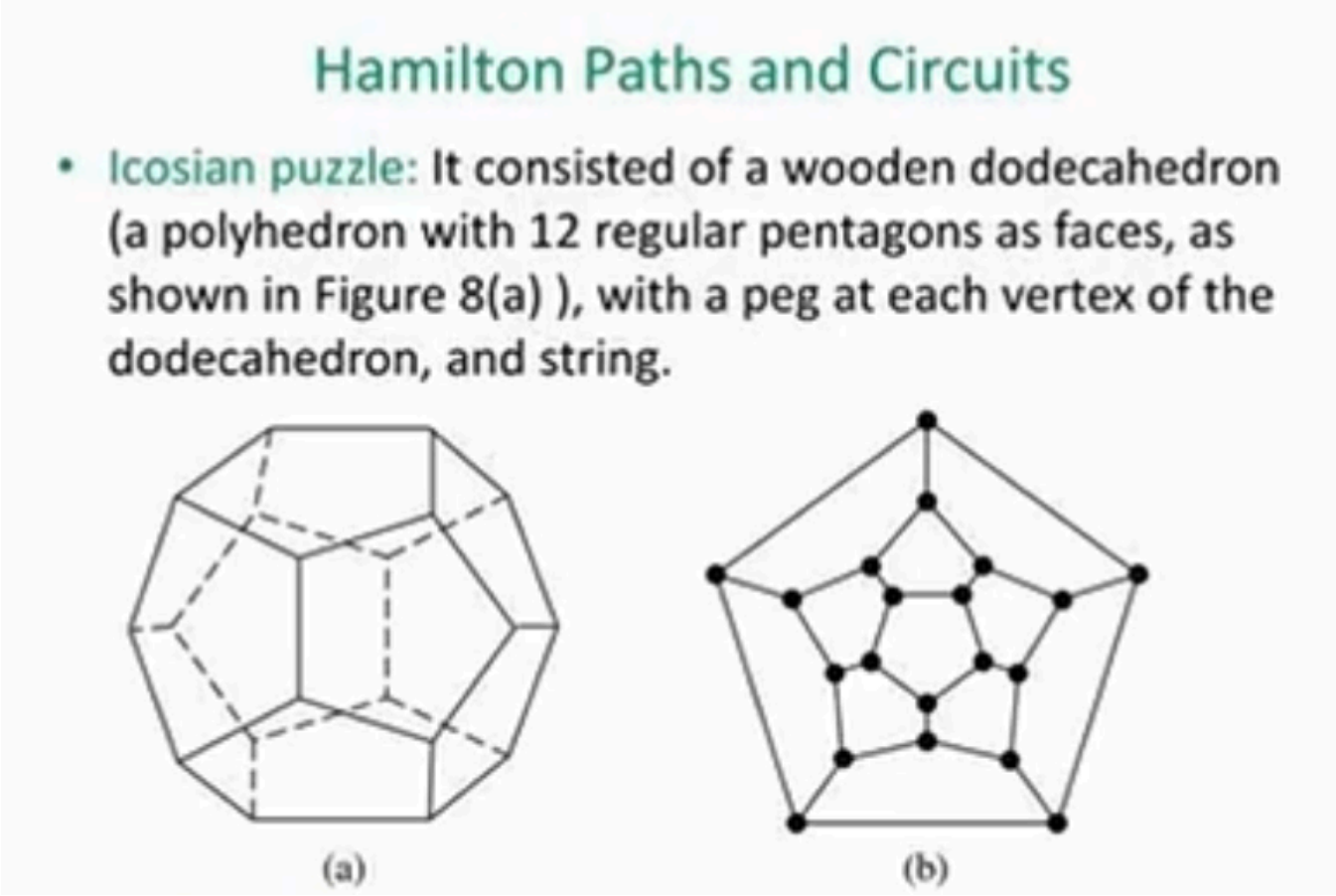
将集合表示为整数：

- 集合中第i个元素是否被选取 → 整数的第i个二进制位是0/1
- 集合运算 → 位运算
- 状态转移 → 对整数（集合）进行运算

二、 引子

1859年，数学家哈密尔顿(Hamilton)提出了一个叫做“周游世界” 的游戏:

在一个正十二面体的20个顶点上，依次标注了伦敦、巴黎、莫斯科等世界上著名的大城市。要求游戏者从某个城市出发，把所有的城市都走过一次，且仅走过一次，然后回到出发点。这类问题就是图论中著名的“哈密尔顿问题”



2.1 Hamilton 问题

给定一个有权无向图，包括n个点，标记为0~n-1，以及连接n个点的边，求从起点0到终点n-1的最短路径。要求必须经过所有点，而且只经过一次。1≤n≤20。

输出:一个整数，表示最短Hamilton路径的长度。

在图论中，遍历图中每个顶点一次且仅一次的路线称为**哈密尔顿路径**，遍历图中每个顶点一次且仅一次的回路(从哪里出发再回到哪里)称为**哈密尔顿回路**。具有哈密尔顿回路的图叫做**哈密尔顿图**。

但是没有一个方法，像“一笔画问题”一样，来判定一个图是否为哈密尔顿图。

从计算复杂性的角度来讲，判定一个图是否为欧拉图是P问题，而判定一个图是否为哈密尔顿图却是NP问题，没有多项式复杂度的解法。

暴力解法:枚举n个点的全排列，共n!个全排列。一个全排列就是一条路径，计算每个全排列的路径长度，需要做n次加法。在所有路径中找最短的路径，总复杂度是O(nxn!).

2.2 DP求解 Hamilton 问题

如果用状态压缩DP求解，能把复杂度降低到$O(n^2x2^n)$。当n=20时$O(n^2x2^{20})$≈4亿，比暴力法好很多。

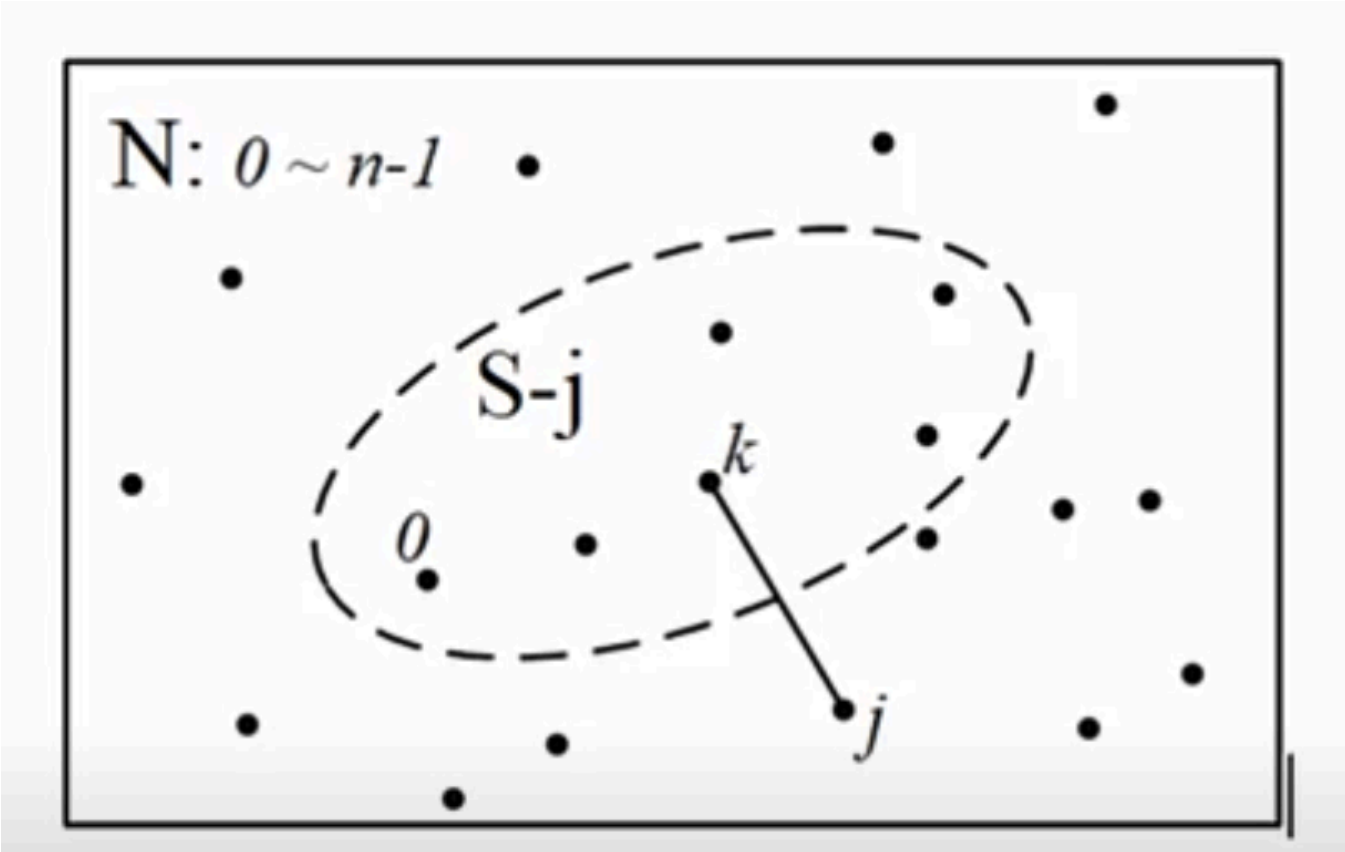
首先定义dp状态，设S是图的一个子集dp[S][j]:表示“集合S内的最短Hamilton路径"，即从起点0出发经过s中所有点，到达终点j时的最短路径;集合S中包括j点。
让S从最小的子集逐步扩展到整个图，最后得到的dp[N][n-1]就是答案，N表示包含图上所有点的集合。

如何求dp[S][j]?

可以从小问题S-j递推到大问题S。其中S-j表示从集合S中去掉j，即不包含j点的集合。

如何从S-j递推到S?

设k是S-j中一个点，把从0到j的路径分为两部分:(0-1...-k)和k-(k+1)-...-j。以k为变量枚举S-j中所有的点，找出最短的路径



状态转移方程为 `dp[S][j]=min{dp[S-j][k]+dist(k,j)}, k∈S-j`

集合S初始时只包含起点0，然后逐步将图中的点包含进来，直到最后包含所有点。这个过程用状态转移方程实现。

2.3 用状态压缩DP编码

如何操作集合S?

状态压缩DP的技巧:用一个二进制数表示集合S，即把S“压缩"到一个二进制数中。

0表示S不包含这个点，等于1表示包含。S的每一位表示图上的1个点，等于例如S=00000101，其中有两个1,表示集合中包含点2、0。

```
    for(int S=1; S<(1<<n); S++)    //从小集合扩展到大集合，集合用S的二进制表示
        for(int j=0; j<n; j++)     //枚举点j
            if((S>>j) & 1)         //(1): 这个判断与下面的(2)一起起作用
                for(int k=0; k<n; k++)        //枚举到达j的点k，k属于集合S-j
                    if((S^(1<<j)) >> k & 1)   //(2): k属于集合S-j。S-j用(1)保证
                  //把(1)和(2)写在一起，像下面这样，更容易理解，但是效率低一点:
                  //if( ((S>>j) & 1) && ((S^(1<<j)) >> k & 1) )
                        dp[S][j] = min(dp[S][j],dp[S^(1<<j)][k] + dist[k][j]);
}
```

三、 状态压缩DP的原理

应用背景:以集合内的元素为状态，且集合一般用二进制来表示和处理。所以又可以称为“集合DP”(DP on Subsets)。

集合问题一般是指数复杂度的(NP问题)，例如:

(1)子集问题，设元素无先后关系，那么共有2^n个子集;

(2)排列问题，对所有h个元素进行全排列，共有n!个全排列。

状态压缩DP的思想:集合的状态(子集或排列)，如果用二进制表示状态，并用二进制的位运算来遍历和操作，又简单又快。

当然，由于集合问题是NP问题，所以状态压缩DP的复杂度仍然是指数级的，只能用于小规模问题的求解。

一个问题能用状态压缩DP求解，时间复杂度主要取决于DP算法，与是否使用状态压缩关系不大。状态压缩只是DP处理集合的工具，也可以用其他工具处理集合，只是不太方面，时间复杂度稍高一些。